示例客户 —— 响应头配置体检报告

生成时间：2026-11-01T11:56:15Z ｜ 工具版本：unknown ｜ https_hint：auto ｜ 输入：3 项（成功 3 / 失败 0）

结论

3 份报文共 35 条信号：2 条要立刻处理、14 条尽快修、8 条可选加固、9 条加固建议（有空再修）、2 条可忽略。五项相加等于总数（8 + 9 = 17 条 optional）；下面按影响排序列出该动手的部分。

现在就要动手（16 条）

排序口径：按实际后果排（must-fix 先于 should-fix，同级按责任方），不按 severity 字母序，也不按命中数。标记「配置矛盾（功能问题）」的条目属于配置自相矛盾、后果是功能失效。

#	问题	谁改	改什么	怎么验证	不改会怎样	命中
1	`cookie.missing-secure`	后端	第一步：先把站点切到 HTTPS 并下发 HSTS——否则这条改不了：Strict Secure Cookies 下，非安全源上带 Secure 的 Cookie 会被浏览器拒绝，照做会直接丢登录态；第二步：切到 HTTPS 之后再加 Secure 属性，例如 Set-Cookie: sid=...; Secure; HttpOnly; SameSite=Lax	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i secure`	Cookie 会在明文链路上被发送，同网段任何人都能直接读走登录态。	2
2	`cookie.missing-httponly`	后端	在 Set-Cookie 里补 HttpOnly 属性，例如 `Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i httponly`	脚本能读到这枚 Cookie，一次 XSS 就能把登录态带走。	3
3	`cookie.missing-samesite`	后端	在 Set-Cookie 里补 `SameSite=Lax`（需要跨站携带时用 `SameSite=None; Secure`）	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i samesite`	别的站点发起请求时浏览器会自动带上这枚 Cookie，可被用于跨站请求伪造。	2
4	`header.cache-on-sensitive`	后端	对含会话或敏感字段的响应加 `Cache-Control: no-store`（可配 `Pragma: no-cache`）	`curl -sI https://<域名>/ | grep -i cache-control`	中间的共享缓存或下一个用户可能拿到这份内容，造成越权读取。	1
5	`header.csp-missing`	后端	第一步：把策略先以 Content-Security-Policy-Report-Only 上线，观察并收集违规，直到每条违规都能说出它来自哪个第三方脚本或哪段内联代码（不需要清零，需要能解释来源）；第二步：按「只有 Report-Only」那条的样子做——把现有 RO 的策略串复制成强制版，先在一个页面上启用（单页灰度），确认无白屏再扩大；第三步：单页稳定后再全量切换，并保留 Report-Only 继续收集新违规。注意：此时报告会把本项降为「有空再修」（csp-	`curl -s I https://<域名>/ | grep -i '^content-security-policy'`	没有任何来源限制，一旦页面存在注入点，脚本可直接从任意外部域加载执行。	1

#	问题	谁改	改什么	怎么验证	不改会怎样	命中
			report-only-only）——这是迁移中的正常状态，不是问题已解决			
6	`header.csp-wildcard-allowlist`	后端	命中的是 `*` / https://* / http://* / https: 这类「任意主机」来源。唯一收敛方向：换成具体主机名列表，或对脚本改用 nonce/hash	`curl -sI https://<域名>/ | grep -i '^content-security-policy' | grep -E '(default-src|script-src|object-src) [^;]*\*'`	`*` 允许任意来源，白名单形同虚设，XSS 的外部载入通道仍然是通的。	1
7	`header.missing-coop`	后端	先判断：你的页面会被**别的站点**开窗或弹窗调用吗（OAuth 回调、嵌入式登录）？不会被调用 → 用 Cross-Origin-Opener-Policy: same-origin（最简单，直接切断 window.opener）；会被调用 → 用 same-origin-allow-popups，并让对端改用 postMessage 传值。（原文保留如下）响应头加 Cross-Origin-Opener-Policy: same-origin；需要被别的站点开窗调用的页面（OAuth 回调、嵌入式登录）改用 same-origin-allow-popups，并说明：OAuth / 支付回调这类 openers 场景（页面被第三方开窗/弹窗被调用）需要与对端确认用 postMessage 传值；同一站点下的多个子域（如 a.example.com / b.example.com）要逐个单独评估，不能一把改成 same-origin。不要退回到 unsafe-none：它让打开的窗口继续与本页共享浏览上下文，等于关闭了这项保护（与不设这个头等价）。边界情况：如果你的页面确实被别人跨域开窗、且依赖 window.opener 回传结果，这一项在当前架构下没有既安全又可用的取值（same-origin 与 same-origin-allow-popups 都会切断回传）——此时正确的做法是拆分独立子域或改用 postMessage 传值，而不是退回 unsafe-none	`curl -sI https://<域名>/ | grep -i '^cross-origin-opener-policy'`	同源打开的窗口之间仍共享浏览上下文，window.opener 可被用来做跨站脚本注入。	2
8	`header.missing-frame-options`	后端	响应加 `X-Frame-Options: DENY`（或 CSP `frame-ancestors 'none'`）	`curl -sI https://<域名>/ | grep -iE 'x-frame-options|frame-ancestors'`	任何站点都能把这页嵌进 iframe，诱导已登录用户点击（点击劫持）。	1
9	`header.missing-nosniff`	后端	响应加 `X-Content-Type-Options: nosniff`	`curl -sI https://<域名>/ | grep -i x-content-type-options`	浏览器可能把上传的文本文件当脚本执行，上传功能等于半个 XSS 入口。	1
10	`header.weak-csp`	后端	把 CSP 里的 `'unsafe-inline'` 换成 nonce/hash（如 `script-src 'nonce-<随机串>'`），把	`curl -sI https://<域名>/ |`	CSP 对注入脚本基本不起拦截作	2

#	问题	谁改	改什么	怎么验证	不改会怎样	命中
			`'unsafe-eval'` 去掉，把单独的 `*` 收窄成明确来源	`grep -i content-security-policy`	用，XSS 一旦出现就等于没有防线。	

（本表只列前 10 条；其余 6 条按规则见下方各输入小节与 --json）

可以忽略（2 条）

共 2 条，属于加固建议或统计类 cookie 的正常配置，不影响可用性；明细见下方各输入小节。其中 0 条来自分析类 cookie（_ga / _gid / _fbp / _hjid / _clck）的过期时间，不计入风险。

有空再修（9 条）

共 9 条，按规则分行说明为什么改了更好但非必须；明细见下方各输入小节。

可选加固（8 条）

这一组按「是否需要独立的整改排期」划分：通常是加固建议，不构成立即被攻破的路径；明细见下方各输入小节。

摘要

指标	值
输入文件数	3
成功 / 失败	3 / 0
信号总数	35
severity：high	0
severity：medium	5
severity：low	17
severity：info	13
category：cookie	17
category：response-header	18
category：request	0
格式：har	0
格式：curl	0
格式：burp	0
格式：raw	3
格式：legacy	0
severity 口径	high = 已可被利用；medium = 配置错误或需人工确认；low = 加固建议；info = 提示

聚合视图

< td>有空再修

规则	目标	命中	涉及文件	最高 severity	处置	谁改	改什么
`cookie.missing-secure`	sid	1	2	medium	现在就要动手	后端	第一步：先把站点切到 HTTPS 并下发 HSTS——否则这条改不了：Strict Secure Cookies 下，非安全源上带 Secure 的 Cookie 会被浏览器拒绝，照做会直接丢登录态；第二步：切到 HTTPS 之后再加 Secure 属性，例如 Set-Cookie: sid=...; Secure; HttpOnly; SameSite=Lax
`cookie.missing-secure`	locale	1	2	low	现在就要动手	后端	第一步：先把站点切到 HTTPS 并下发 HSTS——否则这条改不了：Strict Secure Cookies 下，非安全源上带 Secure 的 Cookie 会被浏览器拒绝，照做会直接丢登录态；第二步：切到 HTTPS 之后再加 Secure 属性，例如 Set-Cookie: sid=...; Secure; HttpOnly; SameSite=Lax
`cookie.missing-httponly`	sid	1	3	medium	尽快修	后端	在 Set-Cookie 里补 HttpOnly 属性，例如 `Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`
`cookie.missing-httponly`	__Host-js_csrf	1	3	low	多数场景可忽略	后端	不要给这枚 Cookie 加 HttpOnly：CSRF token 必须能被前端 JS 读取才能回显到请求头，加了会打断 CSRF 防护
`cookie.missing-httponly`	_octo	1	3	low	尽快修	后端	在 Set-Cookie 里补 HttpOnly 属性，例如 `Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`
`cookie.missing-httponly`	locale	1	3	low	尽快修	后端	在 Set-Cookie 里补 HttpOnly 属性，例如

规则	目标	命中	涉及文件	最高 severity	处置	谁改	改什么
							`Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`
`cookie.missing-samesite`	locale	1	2	low	尽快修	后端	在 Set-Cookie 里补 `SameSite=Lax`（需要跨站携带时用 `SameSite=None; Secure`）
`cookie.missing-samesite`	sid	1	2	low	尽快修	后端	在 Set-Cookie 里补 `SameSite=Lax`（需要跨站携带时用 `SameSite=None; Secure`）
`header.cache-on-sensitive`	Set-Cookie	1	1	low	尽快修	后端	对含会话或敏感字段的响应加 `Cache-Control: no-store`（可配 `Pragma: no-cache`）
`header.csp-missing`	Content-Security-Policy	1	1	medium	尽快修	后端	第一步：把策略先以 Content-Security-Policy-Report-Only 上线，观察并收集违规，直到每条违规都能说出它来自哪个第三方脚本或哪段内联代码（不需要清零，需要能解释来源）；第二步：按「只有 Report-Only」那条的样子做——把现有 RO 的策略串复制成强制版，先在一个页面上启用（单页灰度），确认无白屏再扩大；第三步：单页稳定后再全量切换，并保留 Report-Only 继续收集新违规。注意：此时报告会把本项降为「有空再修」（csp-report-only-only）——这是迁移中的正常状态，不是问题已解决
`header.csp-wildcard-allowlist`	Content-Security-Policy	1	1	info	尽快修	后端	命中的是 `*` / https://* / http://* / https: 这类「任意主机」来源。唯一收敛方向：换成具体主机

规则	目标	命中	涉及文件	最高 severity	处置	谁改	改什么
							名列表，或对脚本改用 nonce/hash
`header.missing-coop`	Cross-Origin-Opener-Policy	2	2	low	尽快修	后端	先判断：你的页面会被**别的站点**开窗或弹窗调用吗（OAuth 回调、嵌入式登录）？不会被调用 → 用 Cross-Origin-Opener-Policy: same-origin（最简单，直接切断 window.opener）；会被调用 → 用 same-origin-allow-popups，并让对端改用 postMessage 传值。（原文保留如下）响应头加 Cross-Origin-Opener-Policy: same-origin；需要被别的站点开窗调用的页面（OAuth 回调、嵌入式登录）改用 same-origin-allow-popups，并说明：OAuth / 支付回调这类 openers 场景（页面被第三方开窗/弹窗被调用）需要与对端确认用 postMessage 传值；同一站点下的多个子域（如 a.example.com / b.example.com）要逐个单独评估，不能一把改成 same-origin。不要退回到 unsafe-none：它让打开的窗口继续与本页共享浏览上下文，等于关闭了这项保护（与不设这个头等价）。边界情况：如果你的页面确实被别人跨域开窗、且依赖 window.opener 回传结果，这一项在当前架构下没有既安全又可用的取值（same-origin 与 same-origin- allow-popups 都会切断回传）——

规则	目标	命中	涉及文件	最高 severity	处置	谁改	改什么
							此时正确的做法是拆分独立子域或改用 postMessage 传值，而不是退回 unsafe-none
`header.missing-frame-options`	X-Frame-Options	1	1	medium	尽快修	后端	响应加 `X-Frame-Options: DENY`（或 CSP `frame-ancestors 'none'`）
`header.missing-nosniff`	X-Content-Type-Options	1	1	low	尽快修	后端	响应加 `X-Content-Type-Options: nosniff`
`header.weak-csp`	Content-Security-Policy	2	2	medium	尽快修	后端	把 CSP 里的 `'unsafe-inline'` 换成 nonce/hash（如 `script-src 'nonce-<随机串>'`），把 `'unsafe-eval'` 去掉，把单独的 `*` 收窄成明确来源
`cookie.expires-too-long`	会话身份类 cookie	3	2	low	有空再修	后端	把 Max-Age 收到 30 天以内，或改为会话 Cookie（不写 Max-Age/Expires）
`cookie.expires-too-long`	其它 cookie	3	2	low	有空再修	后端	建议按业务需要收紧有效期（例如 30 天以内）
`cookie.expires-too-long`	统计分析类 cookie	1	2	low	多数场景可忽略	产品或运营	统计/分析类 cookie 保留长期有效期即可，无需调整
`cookie.session-cookie-on-auth`	_gh_sess	1	2	info	有空再修	后端	按需补 `Max-Age=86400`（要持久登录）或明确保持会话 Cookie；关键操作要求二次验证
`cookie.session-cookie-on-auth`	sid	1	2	info	有空再修	后端	按需补 `Max-Age=86400`（要持久登录）或明确保持会话 Cookie；关键操作要求二次验证
`header.missing-coep`	Cross-Origin-Embedder-Policy	3	3	info	后端	注意 require-corp 会打断未声明 CORP/CORS 的第三方资源加载（图片/脚本/iframe 会缺）。**先判断你要不要跨源隔离	

规则	目标	命中	涉及文件	最高 severity	处置	谁改	改什么
						**：只有需要 SharedArrayBuffer / 高精度计时 / Spectre 类隔离的应用（视频编辑、Web 游戏、WASM 高性能计算）才需要它；普通站点、营销页、博客不需要，加了只会增加页面损坏风险。做法：① 盘点页面上的第三方资源并逐一给它们声明 CORP 或 CORS 头；② 给本页面加 Cross-Origin-Embedder-Policy: require-corp，过渡期可用 credentialless 或先灰度一个页面；③ 观察异常后再扩大。两个注意事项：COEP 要与 COOP 成对配置（只加一个不会进入跨源隔离）；credentialless 在 Safari 上不支持，稳妥起见别把它当唯一方案；被嵌入的 iframe 目标自身仍需 CORP，否则照样加载失败	
`header.missing-permissions-policy`	Permissions-Policy	3	3	info	有空再修	后端	响应头加 Permissions-Policy: camera=(self), microphone=(self), geolocation=(self)，只允许本站使用，再按业务需要逐项放开
`header.missing-referrer-policy`	Referrer-Policy	1	1	info	有空再修	后端	HTML 响应加 `Referrer-Policy: strict-origin-when-cross-origin`
`header.server-version-leak`	Server	1	1	info	有空再修	运维	把 `Server` / `X-Powered-By` 换成不带版本的字符串（如 `Server: web`），或在

规则	目标	命中	涉及文件	最高 severity	处置	谁改	改什么
							反向代理里 `server_tokens off`
`header.x-xss-protection-legacy`	X-Xss-Protection	1	1	info	有空再修	运维	显式写成 X-XSS-Protection: 0（不要用删除代替关闭：删除不等于关闭，旧内核仍默认启用）

聚合只是索引：下面「输入明细」保留全部 35 条 finding 与逐条证据，未删除任何一条。

输入明细

1. `py_github_head.raw`

输入注记：宽松正文模式：声明与实际不符时降级不抛错（头部一字节不改）

格式：raw ｜ 事务数：1 ｜ 本文件全部信号 8 条（high 0 / medium 0 / low 4 / info 4）

`cookie.expires-too-long` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: _octo=REDACTED; expires=Sun, 31 Oct 2027 13:58:42 GMT; domain=.github.com; path=/; secure; SameSite=Lax`
处置	optional（有空再修）
谁改	后端
改什么	把 Max-Age 收到 30 天以内，或改为会话 Cookie（不写 Max-Age/Expires）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	长期有效会放大被窃取后的窗口，但不属必须动。
依据	「_octo」的 Expires=Sun, 31 Oct 2027 13:58:42 GMT 距固定基准 2026-01-01 超过 400 天（基准固定，保证隔天重跑结论一致）。
参考	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02

`cookie.expires-too-long` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: logged_in=REDACTED; expires=Sun, 31 Oct 2027 13:58:42 GMT; domain=.github.com; path=/; HttpOnly; secure; SameSite=Lax`

字段	值
处置	optional（有空再修）
谁改	后端
改什么	把 Max-Age 收到 30 天以内，或改为会话 Cookie（不写 Max-Age/Expires）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	长期有效会放大被窃取后的窗口，但不属必须动。
依据	「logged_in」的 Expires=Sun, 31 Oct 2027 13:58:42 GMT 距固定基准 2026-01-01 超过 400 天（基准固定，保证隔天重跑结论一致）。
参考	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02

`cookie.missing-httponly` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: _octo=REDACTED; expires=Sun, 31 Oct 2027 13:58:42 GMT; domain=.github.com; path=/; secure; SameSite=Lax`
处置	should-fix（尽快修）
谁改	后端
改什么	在 Set-Cookie 里补 HttpOnly 属性，例如 `Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i httponly`
不改会怎样	脚本能读到这枚 Cookie，一次 XSS 就能把登录态带走。
依据	普通 Cookie「_octo」没有 HttpOnly 属性，脚本可读；名字不像会话凭据，影响相对较低。
参考	RFC 6265bis §4.1.3, CWE-1004

`header.missing-coop` low

字段	值
where	response
transaction	0
header	`Cross-Origin-Opener-Policy`
证据	`HTTP/1.1 200 OK`
处置	should-fix（尽快修）
谁改	后端
改什么	先判断：你的页面会被**别的站点**开窗或弹窗调用吗（OAuth 回调、嵌入式登录）？不会被调用 → 用 Cross-Origin-Opener-Policy: same-origin（最简单，直接切断 window.opener）；会被调用 → 用 same-origin-allow-popups，并让对端改用 postMessage 传值。（原文保留如下）响应头加 Cross-Origin-Opener-Policy: same-origin；需要被别的站点开窗调用的页面（OAuth 回调、嵌入式登录）改用 same-origin-allow-popups，并说明：OAuth / 支付回调这类 openers 场景（页面被第三方开窗/弹窗被调用）需要与对端确认用 postMessage 传值；同一站点下的多个子域（如 a.example.com / b.example.com）要逐个单

字段	值
	独评估，不能一把改成 same-origin。不要退回到 unsafe-none：它让打开的窗口继续与本页共享浏览上下文，等于关闭了这项保护（与不设这个头等价）。边界情况：如果你的页面确实被别人跨域开窗、且依赖 window.opener 回传结果，这一项在当前架构下没有既安全又可用的取值（same-origin 与 same-origin-allow-popups 都会切断回传）——此时正确的做法是拆分独立子域或改用 postMessage 传值，而不是退回 unsafe-none
怎么验证	`curl -sI https://<域名>/ | grep -i '^cross-origin-opener-policy'`
不改会怎样	同源打开的窗口之间仍共享浏览上下文，window.opener 可被用来做跨站脚本注入。
依据	COOP 缺失：同源打开的窗口之间仍共享浏览上下文，攻击者可用窗口引用（window.opener）做跨站脚本注入面。证据取本响应起始行。只按头名与值匹配，未做完整语义解析。
参考	MDN: Cross-Origin-Opener-Policy

`cookie.session-cookie-on-auth` info

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: _gh_sess=REDACTED; path=/; HttpOnly; secure; SameSite=Lax`
处置	optional（有空再修）
谁改	后端
改什么	按需补 `Max-Age=86400`（要持久登录）或明确保持会话 Cookie；关键操作要求二次验证
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	关掉浏览器就要重新登录；若被他人拿到这枚 Cookie，在有效期内可直接冒用。
依据	鉴权相关 Cookie「_gh_sess」既无 Max-Age 也无 Expires：关闭浏览器即失效，可能是有意设计，仅作提示。
参考	RFC 6265bis §4.1.2

`header.missing-coep` info

字段	值
where	response
transaction	0
header	`Cross-Origin-Embedder-Policy`
证据	`HTTP/1.1 200 OK`
处置	optional（有空再修）
谁改	后端
改什么	注意 require-corp 会打断未声明 CORP/CORS 的第三方资源加载（图片/脚本/iframe 会缺）。**先判断你要不要跨源隔离**：只有需要 SharedArrayBuffer / 高精度计时 / Spectre 类隔离的应用（视频编辑、Web 游戏、WASM 高性能计算）才需要它；普通站点、营销页、博客不需要，加了只会增加页面损坏风险。做法：① 盘点页面上的第三方资源并逐一给它们声明 CORP 或 CORS 头；② 给本页面加 Cross-Origin-Embedder-Policy: require-corp，过渡期可用 credentialless 或先灰度一个页面；③ 观察异常后再扩大。两个注意事项：COEP 要与 COOP 成对配置（只加一个不会进入跨源隔离）；credentialless 在 Safari 上不支持，稳妥起见别把它当唯一方案；被嵌入的 iframe 目标自身仍需 CORP，否则照样加载失败

字段	值
怎么验证	`curl -sI https://<域名>/ | grep -iE '^cross-origin-(embedder|opener)-policy'`；同时用浏览器控制台看有无 `CORP/CORS` 报错
不改会怎样	无法声明只加载明确授权的跨域资源，也就用不上依赖隔离的强特性（如 SharedArrayBuffer）。
依据	COEP 缺失：页面无法声明「只加载明确授权的跨域资源」，因此也用不上依赖隔离的强特性（如 SharedArrayBuffer）。证据取本响应起始行。只按头名与值匹配，未做完整语义解析。
参考	MDN: Cross-Origin-Embedder-Policy

`header.missing-permissions-policy` info

字段	值
where	response
transaction	0
header	`Permissions-Policy`
证据	`HTTP/1.1 200 OK`
处置	optional（有空再修）
谁改	后端
改什么	响应头加 Permissions-Policy: camera=(self), microphone=(self), geolocation=(self)，只允许本站使用，再按业务需要逐项放开
怎么验证	`curl -sI https://<域名>/ | grep -i '^permissions-policy'`
不改会怎样	缺这个头不会让跨源 iframe 获得这些能力（camera/microphone/geolocation 的默认允许列表本来就是 self）；真正后果是**无法进一步收紧**——不能用 () 彻底关掉，也不能只放给某个合作方。
依据	该 HTML 响应没有 Permissions-Policy：无法限制摄像头、麦克风、地理位置等浏览器特性。本轮只看有无，不校验允许列表内容。证据取本响应起始行。
参考	MDN: Permissions-Policy

`header.weak-csp` info

字段	值
where	response
transaction	0
header	`Content-Security-Policy`
证据	▶ 证据（4214 字符，点击展开）
处置	should-fix（尽快修）
谁改	后端
改什么	把 CSP 里的 `'unsafe-inline'` 换成 nonce/hash（如 `script-src 'nonce-<随机串>'`），把 `'unsafe-eval'` 去掉，把单独的 `*` 收窄成明确来源
怎么验证	`curl -sI https://<域名>/ | grep -i content-security-policy`
不改会怎样	CSP 对注入脚本基本不起拦截作用，XSS 一旦出现就等于没有防线。
依据	CSP 含弱来源：style-src:unsafe-inline（仅非脚本类指令命中，影响较低）。只按指令名与来源标记匹配，未做完整 CSP 语法解析。

字段	值
参考	CSP Level 3 §6.1, CWE-79

2. `py_dropbox_home.raw`

输入注记：规范化文本含 latin-1 之外的字符，改用 utf-8 编码

输入注记：宽松正文模式：声明与实际不符时降级不抛错（头部一字节不改）

格式：raw ｜ 事务数：1 ｜ 本文件全部信号 14 条（high 0 / medium 1 / low 9 / info 4）

`header.weak-csp` medium

字段	值
where	response
transaction	0
header	`Content-Security-Policy`
证据	▶ 证据（2272 字符，点击展开）
处置	should-fix（尽快修）
谁改	后端
改什么	把 CSP 里的 `'unsafe-inline'` 换成 nonce/hash（如 `script-src 'nonce-<随机串>'`），把 `'unsafe-eval'` 去掉，把单独的 `*` 收窄成明确来源
怎么验证	`curl -sI https://<域名>/ | grep -i content-security-policy`
不改会怎样	CSP 对注入脚本基本不起拦截作用，XSS 一旦出现就等于没有防线。
依据	CSP 含弱来源：script-src:unsafe-eval、script-src:unsafe-inline、style-src:unsafe-inline、style-src:unsafe-eval（高风险指令命中）。只按指令名与来源标记匹配，未做完整 CSP 语法解析。
参考	CSP Level 3 §6.1, CWE-79

`cookie.expires-too-long` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: gvc=REDACTED; Path=/; Expires=Thu, 30 Oct 2031 14:09:42 GMT; HttpOnly; Secure; SameSite=None`
处置	ignore（多数场景可忽略）
谁改	产品或运营
改什么	统计/分析类 cookie 保留长期有效期即可，无需调整
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE '_ga|_gid|_fbp'`
不改会怎样	长期有效属这类 cookie 的常态，不影响安全。
依据	「gvc」的 Expires=Thu, 30 Oct 2031 14:09:42 GMT 距固定基准 2026-01-01 超过 400 天（基准固定，保证隔天重跑结论一致）。
参考	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02

`cookie.expires-too-long` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: t=REDACTED; Path=/; Domain=dropbox.com; Expires=Sun, 31 Oct 2027 14:09:42 GMT; HttpOnly; Secure; SameSite=None`
处置	optional（有空再修）
谁改	后端
改什么	建议按业务需要收紧有效期（例如 30 天以内）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	建议按业务需要收紧有效期。
依据	「t」的 Expires=Sun, 31 Oct 2027 14:09:42 GMT 距固定基准 2026-01-01 超过 400 天（基准固定，保证隔天重跑结论一致）。
参考	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02

`cookie.expires-too-long` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: __Host-js_csrf=REDACTED; Path=/; Expires=Sun, 31 Oct 2027 14:09:42 GMT; Secure; SameSite=None`
处置	optional（有空再修）
谁改	后端
改什么	建议按业务需要收紧有效期（例如 30 天以内）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	建议按业务需要收紧有效期。
依据	「__Host-js_csrf」的 Expires=Sun, 31 Oct 2027 14:09:42 GMT 距固定基准 2026-01-01 超过 400 天（基准固定，保证隔天重跑结论一致）。
参考	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02

`cookie.expires-too-long` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: __Host-ss=REDACTED; Path=/; Expires=Sun, 31 Oct 2027 14:09:42 GMT; HttpOnly; Secure; SameSite=Strict`

字段	值
处置	optional（有空再修）
谁改	后端
改什么	把 Max-Age 收到 30 天以内，或改为会话 Cookie（不写 Max-Age/Expires）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	长期有效会放大被窃取后的窗口，但不属必须动。
依据	「__Host-ss」的 Expires=Sun, 31 Oct 2027 14:09:42 GMT 距固定基准 2026-01-01 超过 400 天（基准固定，保证隔天重跑结论一致）。
参考	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02

`cookie.expires-too-long` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: locale=REDACTED; Path=/; Domain=dropbox.com; Expires=Thu, 30 Oct 2031 14:09:42 GMT`
处置	optional（有空再修）
谁改	后端
改什么	建议按业务需要收紧有效期（例如 30 天以内）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	建议按业务需要收紧有效期。
依据	「locale」的 Expires=Thu, 30 Oct 2031 14:09:42 GMT 距固定基准 2026-01-01 超过 400 天（基准固定，保证隔天重跑结论一致）。
参考	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02

`cookie.missing-httponly` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: __Host-js_csrf=REDACTED; Path=/; Expires=Sun, 31 Oct 2027 14:09:42 GMT; Secure; SameSite=None`
处置	ignore（多数场景可忽略）
谁改	后端
改什么	不要给这枚 Cookie 加 HttpOnly：CSRF token 必须能被前端 JS 读取才能回显到请求头，加了会打断 CSRF 防护
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'csrf|xsrf'`
不改会怎样	加了 HttpOnly 会让前端拿不到 token，表单提交/接口调用直接失败（功能中断）

字段	值
依据	会话类 Cookie（段名命中）「__Host-js_csrf」没有 HttpOnly 属性，脚本可读；名字不像会话凭据，影响相对较低。
参考	RFC 6265bis §4.1.3, CWE-1004

`cookie.missing-httponly` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: locale=REDACTED; Path=/; Domain=dropbox.com; Expires=Thu, 30 Oct 2031 14:09:42 GMT`
处置	should-fix（尽快修）
谁改	后端
改什么	在 Set-Cookie 里补 HttpOnly 属性，例如 `Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i httponly`
不改会怎样	脚本能读到这枚 Cookie，一次 XSS 就能把登录态带走。
依据	普通 Cookie「locale」没有 HttpOnly 属性，脚本可读；名字不像会话凭据，影响相对较低。
参考	RFC 6265bis §4.1.3, CWE-1004

`cookie.missing-samesite` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: locale=REDACTED; Path=/; Domain=dropbox.com; Expires=Thu, 30 Oct 2031 14:09:42 GMT`
处置	should-fix（尽快修）
谁改	后端
改什么	在 Set-Cookie 里补 `SameSite=Lax`（需要跨站携带时用 `SameSite=None; Secure`）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i samesite`
不改会怎样	别的站点发起请求时浏览器会自动带上这枚 Cookie，可被用于跨站请求伪造。
依据	「locale」没有 SameSite 属性，跨站请求可能带上它（浏览器默认策略因版本而异，不能当作已设置）。
参考	RFC 6265bis §4.1.4, CWE-1275

`cookie.missing-secure` low

字段	值
where	response
transaction	0

字段	值
header	`Set-Cookie`
证据	`Set-Cookie: locale=REDACTED; Path=/; Domain=dropbox.com; Expires=Thu, 30 Oct 2031 14:09:42 GMT`
处置	must-fix（现在就要动手）
谁改	后端
改什么	第一步：先把站点切到 HTTPS 并下发 HSTS——否则这条改不了：Strict Secure Cookies 下，非安全源上带 Secure 的 Cookie 会被浏览器拒绝，照做会直接丢登录态；第二步：切到 HTTPS 之后再加 Secure 属性，例如 Set-Cookie: sid=...; Secure; HttpOnly; SameSite=Lax
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i secure`
不改会怎样	Cookie 会在明文链路上被发送，同网段任何人都能直接读走登录态。
依据	普通 Cookie「locale」没有 Secure 属性，在明文信道上可能被发出。
参考	RFC 6265bis §4.1.3, CWE-614

`header.csp-wildcard-allowlist` info

字段	值
where	response
transaction	0
header	`Content-Security-Policy`
证据	▶ 证据（2272 字符，点击展开）
处置	should-fix（尽快修）
谁改	后端
改什么	命中的是 `*` / https://* / http://* / https: 这类「任意主机」来源。唯一收敛方向：换成具体主机名列表，或对脚本改用 nonce/hash
怎么验证	`curl -sI https://<域名>/ | grep -i '^content-security-policy' | grep -E '(default-src|script-src|object-src)[^;]*\*'`
不改会怎样	`*` 允许任意来源，白名单形同虚设，XSS 的外部载入通道仍然是通的。
依据	CSP 里出现通配来源：connect-src:https://*、font-src:https://*、frame-src:https://*、img-src:https://*、media-src:https://*、style-src:https://*（仅非脚本类指令命中，影响较低）。`*` / `https://*` / `http://*` / `https:` 都等于放行任意主机；应改成具体主机名，或改用 nonce/hash。只按指令名与来源标记匹配，未做完整 CSP 语法解析。
参考	MDN: CSP source values（* 通配）

`header.missing-coep` info

字段	值
where	response
transaction	0
header	`Cross-Origin-Embedder-Policy`
证据	`HTTP/1.1 200 OK`
处置	optional（有空再修）
谁改	后端

字段	值
改什么	注意 require-corp 会打断未声明 CORP/CORS 的第三方资源加载（图片/脚本/iframe 会缺）。**先判断你要不要跨源隔离**：只有需要 SharedArrayBuffer / 高精度计时 / Spectre 类隔离的应用（视频编辑、Web 游戏、WASM 高性能计算）才需要它；普通站点、营销页、博客不需要，加了只会增加页面损坏风险。做法：① 盘点页面上的第三方资源并逐一给它们声明 CORP 或 CORS 头；② 给本页面加 Cross-Origin-Embedder-Policy: require-corp，过渡期可用 credentialless 或先灰度一个页面；③ 观察异常后再扩大。两个注意事项：COEP 要与 COOP 成对配置（只加一个不会进入跨源隔离）；credentialless 在 Safari 上不支持，稳妥起见别把它当唯一方案；被嵌入的 iframe 目标自身仍需 CORP，否则照样加载失败
怎么验证	`curl -sI https://<域名>/ | grep -iE '^cross-origin-(embedder|opener)-policy'`；同时用浏览器控制台看有无 `CORP/CORS` 报错
不改会怎样	无法声明只加载明确授权的跨域资源，也就用不上依赖隔离的强特性（如 SharedArrayBuffer）。
依据	COEP 缺失：页面无法声明「只加载明确授权的跨域资源」，因此也用不上依赖隔离的强特性（如 SharedArrayBuffer）。证据取本响应起始行。只按头名与值匹配，未做完整语义解析。
参考	MDN: Cross-Origin-Embedder-Policy

`header.missing-permissions-policy` info

字段	值
where	response
transaction	0
header	`Permissions-Policy`
证据	`HTTP/1.1 200 OK`
处置	optional（有空再修）
谁改	后端
改什么	响应头加 Permissions-Policy: camera=(self), microphone=(self), geolocation=(self)，只允许本站使用，再按业务需要逐项放开
怎么验证	`curl -sI https://<域名>/ | grep -i '^permissions-policy'`
不改会怎样	缺这个头不会让跨源 iframe 获得这些能力（camera/microphone/geolocation 的默认允许列表本来就是 self）；真正后果是**无法进一步收紧**——不能用 () 彻底关掉，也不能只放给某个合作方。
依据	该 HTML 响应没有 Permissions-Policy：无法限制摄像头、麦克风、地理位置等浏览器特性。本轮只看有无，不校验允许列表内容。证据取本响应起始行。
参考	MDN: Permissions-Policy

`header.x-xss-protection-legacy` info

字段	值
where	response
transaction	0
header	`X-Xss-Protection`
证据	`X-Xss-Protection: 1; mode=block`
处置	optional（有空再修）
谁改	运维
改什么	显式写成 X-XSS-Protection: 0（不要用删除代替关闭：删除不等于关闭，旧内核仍默认启用）

字段	值
怎么验证	`curl -sI https://<域名>/ | grep -i '^x-xss-protection'`
不改会怎样	旧版 XSS 过滤器已被现代浏览器移除或忽略，留着不会增加防护，反而在某些旧内核上可被用来放大 XSS。
依据	X-XSS-Protection 处于启用态（值 1）。该头已被现代浏览器移除或忽略，旧版 XSS 过滤器本身可被用于绕过 CSP 或放大 XSS；正确的处理是显式写成 X-XSS-Protection: 0。删除该头不等于关闭（旧内核仍默认启用），因此不把删除当推荐动作。
参考	OWASP Secure Headers Project: X-XSS-Protection

3. `wire_gzip-session.raw`

输入注记：源文件不是合法 UTF-8，按 latin-1 解码（保留字节）

输入注记：宽松正文模式：声明与实际不符时降级不抛错（头部一字节不改）

输入注记：事务 0：正文为 gzip 压缩，未解压（证据保持原样，正文类判据不适用）

格式：raw | 事务数：1 | 本文件全部信号 13 条（high 0 / medium 4 / low 4 / info 5）

`cookie.missing-httponly` medium

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: sid=REDACTED; Path=/`
处置	should-fix（尽快修）
谁改	后端
改什么	在 Set-Cookie 里补 HttpOnly 属性，例如 `Set-Cookie: sid=...; HttpOnly; Secure; SameSite=Lax`
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i httponly`
不改会怎样	脚本能读到这枚 Cookie，一次 XSS 就能把登录态带走。
依据	会话类 Cookie（显式名单）「sid」没有 HttpOnly 属性，脚本可读；名字属会话凭据，一旦被脚本读取影响更大。
参考	RFC 6265bis §4.1.3, CWE-1004

`cookie.missing-secure` medium

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: sid=REDACTED; Path=/`
处置	must-fix（现在就要动手）
谁改	后端
改什么	第一步：先把站点切到 HTTPS 并下发 HSTS——否则这条改不了：Strict Secure Cookies 下，非安全源上带 Secure 的 Cookie 会被浏览器拒绝，照做会直接丢登录态；第二步：切到 HTTPS 之后再加 Secure 属性，例如 Set-Cookie: sid=...; Secure; HttpOnly;

字段	值
	SameSite=Lax
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i secure`
不改会怎样	Cookie 会在明文链路上被发送，同网段任何人都能直接读走登录态。
依据	会话类 Cookie（显式名单）「sid」没有 Secure 属性，在明文信道上可能被发出。
参考	RFC 6265bis §4.1.3, CWE-614

`header.csp-missing` medium

字段	值
where	response
transaction	0
header	`Content-Security-Policy`
证据	`HTTP/1.1 200 OK`
处置	should-fix（尽快修）
谁改	后端
改什么	第一步：把策略先以 Content-Security-Policy-Report-Only 上线，观察并收集违规，直到每条违规都能说出它来自哪个第三方脚本或哪段内联代码（不需要清零，需要能解释来源）；第二步：按「只有 Report-Only」那条的样子做——把现有 RO 的策略串复制成强制版，先在一个页面上启用（单页灰度），确认无白屏再扩大；第三步：单页稳定后再全量切换，并保留 Report-Only 继续收集新违规。注意：此时报告会把本项降为「有空再修」（csp-report-only-only）——这是迁移中的正常状态，不是问题已解决
怎么验证	`curl -sI https://<域名>/ | grep -i '^content-security-policy'`
不改会怎样	没有任何来源限制，一旦页面存在注入点，脚本可直接从任意外部域加载执行。
依据	该 HTML 响应没有可用的 Content-Security-Policy。浏览器不会限制任何脚本/样式/框架来源。证据取本响应起始行。
参考	MDN: Content-Security-Policy

`header.missing-frame-options` medium

字段	值
where	response
transaction	0
header	`X-Frame-Options`
证据	`HTTP/1.1 200 OK`
处置	should-fix（尽快修）
谁改	后端
改什么	响应加 `X-Frame-Options: DENY`（或 CSP `frame-ancestors 'none'`）
怎么验证	`curl -sI https://<域名>/ | grep -iE 'x-frame-options|frame-ancestors'`
不改会怎样	任何站点都能把这页嵌进 iframe，诱导已登录用户点击（点击劫持）。
依据	该头整行不存在（X-Frame-Options），强制 CSP 里也没有 frame-ancestors：缺少帧嵌入限制信号。证据取本响应起始行。

字段	值
参考	OWASP Secure Headers, CWE-1021

`cookie.missing-samesite` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: sid=REDACTED; Path=/`
处置	should-fix（尽快修）
谁改	后端
改什么	在 Set-Cookie 里补 `SameSite=Lax`（需要跨站携带时用 `SameSite=None; Secure`）
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -i samesite`
不改会怎样	别的站点发起请求时浏览器会自动带上这枚 Cookie，可被用于跨站请求伪造。
依据	「sid」没有 SameSite 属性，跨站请求可能带上它（浏览器默认策略因版本而异，不能当作已设置）。
参考	RFC 6265bis §4.1.4, CWE-1275

`header.cache-on-sensitive` low

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: sid=REDACTED; Path=/`
处置	should-fix（尽快修）
谁改	后端
改什么	对含会话或敏感字段的响应加 `Cache-Control: no-store`（可配 `Pragma: no-cache`）
怎么验证	`curl -sI https://<域名>/ | grep -i cache-control`
不改会怎样	中间的共享缓存或下一个用户可能拿到这份内容，造成越权读取。
依据	响应下发了会话类 Cookie「sid」（显式名单），但 Cache-Control 里没有 no-store，可能被共享缓存留存。
参考	OWASP WSTG-ATHN-06, CWE-525

`header.missing-coop` low

字段	值
where	response
transaction	0
header	`Cross-Origin-Opener-Policy`

字段	值
证据	`HTTP/1.1 200 OK`
处置	should-fix（尽快修）
谁改	后端
改什么	先判断：你的页面会被**别的站点**开窗或弹窗调用吗（OAuth 回调、嵌入式登录）？不会被调用 → 用 Cross-Origin-Opener-Policy: same-origin（最简单，直接切断 window.opener）；会被调用 → 用 same-origin-allow-popups，并让对端改用 postMessage 传值。（原文保留如下）响应头加 Cross-Origin-Opener-Policy: same-origin；需要被别的站点开窗调用的页面（OAuth 回调、嵌入式登录）改用 same-origin-allow-popups，并说明：OAuth / 支付回调这类 openers 场景（页面被第三方开窗/弹窗被调用）需要与对端确认用 postMessage 传值；同一站点下的多个子域（如 a.example.com / b.example.com）要逐个单独评估，不能一把改成 same-origin。不要退回到 unsafe-none：它让打开的窗口继续与本页共享浏览上下文，等于关闭了这项保护（与不设这个头等价）。边界情况：如果你的页面确实被别人跨域开窗、且依赖 window.opener 回传结果，这一项在当前架构下没有既安全又可用的取值（same-origin 与 same-origin-allow-popups 都会切断回传）——此时正确的做法是拆分独立子域或改用 postMessage 传值，而不是退回 unsafe-none
怎么验证	`curl -sI https://<域名>/ | grep -i '^cross-origin-opener-policy'`
不改会怎样	同源打开的窗口之间仍共享浏览上下文，window.opener 可被用来做跨站脚本注入。
依据	COOP 缺失：同源打开的窗口之间仍共享浏览上下文，攻击者可用窗口引用（window.opener）做跨站脚本注入面。证据取本响应起始行。只按头名与值匹配，未做完整语义解析。
参考	MDN: Cross-Origin-Opener-Policy

`header.missing-nosniff` low

字段	值
where	response
transaction	0
header	`X-Content-Type-Options`
证据	`HTTP/1.1 200 OK`
处置	should-fix（尽快修）
谁改	后端
改什么	响应加 `X-Content-Type-Options: nosniff`
怎么验证	`curl -sI https://<域名>/ | grep -i x-content-type-options`
不改会怎样	浏览器可能把上传的文本文件当脚本执行，上传功能等于半个 XSS 入口。
依据	该头整行不存在（X-Content-Type-Options: nosniff），浏览器可能自行嗅探类型。证据取本响应起始行。
参考	OWASP Secure Headers, CWE-436

`cookie.session-cookie-on-auth` info

字段	值
where	response
transaction	0
header	`Set-Cookie`
证据	`Set-Cookie: sid=REDACTED; Path=/`

字段	值
处置	optional（有空再修）
谁改	后端
改什么	按需补 `Max-Age=86400`（要持久登录）或明确保持会话 Cookie；关键操作要求二次验证
怎么验证	`curl -sI https://<域名>/ | grep -i '^set-cookie' | grep -iE 'max-age|expires'`
不改会怎样	关掉浏览器就要重新登录；若被他人拿到这枚 Cookie，在有效期内可直接冒用。
依据	鉴权相关 Cookie「sid」既无 Max-Age 也无 Expires：关闭浏览器即失效，可能是有意设计，仅作提示。
参考	RFC 6265bis §4.1.2

`header.missing-coep` info

字段	值
where	response
transaction	0
header	`Cross-Origin-Embedder-Policy`
证据	`HTTP/1.1 200 OK`
处置	optional（有空再修）
谁改	后端
改什么	注意 require-corp 会打断未声明 CORP/CORS 的第三方资源加载（图片/脚本/iframe 会缺）。**先判断你要不要跨源隔离**：只有需要 SharedArrayBuffer / 高精度计时 / Spectre 类隔离的应用（视频编辑、Web 游戏、WASM 高性能计算）才需要它；普通站点、营销页、博客不需要，加了只会增加页面损坏风险。做法：① 盘点页面上的第三方资源并逐一给它们声明 CORP 或 CORS 头；② 给本页面加 Cross-Origin-Embedder-Policy: require-corp，过渡期可用 credentialless 或先灰度一个页面；③ 观察异常后再扩大。两个注意事项：COEP 要与 COOP 成对配置（只加一个不会进入跨源隔离）；credentialless 在 Safari 上不支持，稳妥起见别把它当唯一方案；被嵌入的 iframe 目标自身仍需 CORP，否则照样加载失败
怎么验证	`curl -sI https://<域名>/ | grep -iE '^cross-origin-(embedder|opener)-policy'`；同时用浏览器控制台看有无 `CORP/CORS` 报错
不改会怎样	无法声明只加载明确授权的跨域资源，也就用不上依赖隔离的强特性（如 SharedArrayBuffer）。
依据	COEP 缺失：页面无法声明「只加载明确授权的跨域资源」，因此也用不上依赖隔离的强特性（如 SharedArrayBuffer）。证据取本响应起始行。只按头名与值匹配，未做完整语义解析。
参考	MDN: Cross-Origin-Embedder-Policy

`header.missing-permissions-policy` info

字段	值
where	response
transaction	0
header	`Permissions-Policy`
证据	`HTTP/1.1 200 OK`
处置	optional（有空再修）
谁改	后端
改什么	响应头加 Permissions-Policy: camera=(self), microphone=(self), geolocation=(self)，只允许本站使用，再按业务需要逐项放开

字段	值
怎么验证	`curl -sI https://<域名>/ | grep -i '^permissions-policy'`
不改会怎样	缺这个头不会让跨源 iframe 获得这些能力（camera/microphone/geolocation 的默认允许列表本来就是 self）；真正后果是**无法进一步收紧**——不能用 () 彻底关掉，也不能只放给某个合作方。
依据	该 HTML 响应没有 Permissions-Policy：无法限制摄像头、麦克风、地理位置等浏览器特性。本轮只看有无，不校验允许列表内容。证据取本响应起始行。
参考	MDN: Permissions-Policy

`header.missing-referrer-policy` info

字段	值
where	response
transaction	0
header	`Referrer-Policy`
证据	`HTTP/1.1 200 OK`
处置	optional（有空再修）
谁改	后端
改什么	HTML 响应加 `Referrer-Policy: strict-origin-when-cross-origin`
怎么验证	`curl -sI https://<域名>/ | grep -i referrer-policy`
不改会怎样	跳转到第三方时可能把带查询参数的完整 URL 一起送出去（可能含 token）。注意方向：不设时现代浏览器默认 strict-origin-when-cross-origin（安全），本项只是少一层显式声明；把值写成不安全的值反而更糟 → 见 referrer-policy-unsafe。
依据	该头整行不存在（Referrer-Policy），跨站跳转会按浏览器默认策略携带 Referer。证据取本响应起始行。
参考	OWASP Secure Headers

`header.server-version-leak` info

字段	值
where	response
transaction	0
header	`Server`
证据	`Server: BaseHTTP Python/3.15.2`
处置	optional（有空再修）
谁改	运维
改什么	把 `Server` / `X-Powered-By` 换成不带版本的字符串（如 `Server: web`），或在反向代理里 `server_tokens off`
怎么验证	`curl -sI https://<域名>/ | grep -iE '^(server|x-powered-by)'`
不改会怎样	攻击者能直接对号入座已知版本漏洞，省掉一轮探测。
依据	Server 值中含版本号模式「3.15」，可辅助指纹识别（仅信号，未做版本漏洞比对）。
参考	OWASP WSTG-INFO-02

证据完整性

无证据完整性告警

规则覆盖

状态	条数
hit	29
clean	40
not-applicable	24
skipped	6

skipped 原因（逐条）

输入	规则	原因
`py_github_head.raw`	`header.missing-hsts`	传输方式未知（`https_hint=None`），无法判断 `HSTS` 是否生效
`py_github_head.raw`	`header.hsts-short-max-age`	传输方式未知（`https_hint=None`），无法判断 `HSTS` 是否生效
`py_dropbox_home.raw`	`header.missing-hsts`	传输方式未知（`https_hint=None`），无法判断 `HSTS` 是否生效
`py_dropbox_home.raw`	`header.hsts-short-max-age`	传输方式未知（`https_hint=None`），无法判断 `HSTS` 是否生效
`wire_gzip-session.raw`	`header.missing-hsts`	传输方式未知（`https_hint=None`），无法判断 `HSTS` 是否生效
`wire_gzip-session.raw`	`header.hsts-short-max-age`	传输方式未知（`https_hint=None`），无法判断 `HSTS` 是否生效

附录

规则清单

rule_id	标题	依据
`cookie.missing-httponly`	Set-Cookie 缺少 HttpOnly 属性	RFC 6265bis §4.1.3, CWE-1004
`cookie.missing-secure`	Set-Cookie 缺少 Secure 属性	RFC 6265bis §4.1.3, CWE-614
`cookie.missing-samesite`	Set-Cookie 缺少 SameSite 属性	RFC 6265bis §4.1.4, CWE-1275
`cookie.samesite-none-without-secure`	SameSite=None 但缺少 Secure	RFC 6265bis §4.1.4, CWE-1275
`cookie.domain-too-wide`	Cookie Domain 过宽（单标签域）	RFC 6265bis §4.1.2.3, CWE-1275
`cookie.expires-too-long`	Cookie 有效期过长	RFC 6265bis §4.1.2.2, OWASP WSTG-SESS-02
`cookie.prefix-violation`	Cookie 名称前缀约束被违反	RFC 6265bis §4.1.3, CWE-1004
`cookie.session-cookie-on-auth`	鉴权相关 Cookie 无持久化属性（会话 Cookie）	RFC 6265bis §4.1.2
`header.missing-hsts`	HTTPS 响应缺少 Strict-Transport-Security	RFC 6797 §6.1, OWASP Secure Headers
`header.hsts-short-max-age`	HSTS max-age 短于 180 天	RFC 6797 §6.1.1
`header.missing-nosniff`	缺少 X-Content-Type-Options: nosniff	OWASP Secure Headers, CWE-436

rule_id	标题	依据
`header.missing-frame-options`	缺少有效帧保护（X-Frame-Options / CSP frame-ancestors）	OWASP Secure Headers, CWE-1021
`header.weak-csp`	CSP 含弱指令（unsafe-inline / unsafe-eval / 单独 *）	CSP Level 3 §6.1, CWE-79
`header.cors-wildcard-with-credentials`	CORS 通配来源与凭据同时出现	Fetch Standard §3.2, CWE-942
`header.cors-reflect-origin`	CORS 反射请求 Origin 并放行凭据	Fetch Standard §3.2, CWE-942
`header.cors-null-origin`	CORS 允许 null 来源	Fetch Standard §3.2, CWE-942
`header.server-version-leak`	Server / X-Powered-By 泄露版本号	OWASP WSTG-INFO-02
`header.missing-referrer-policy`	缺少 Referrer-Policy	OWASP Secure Headers
`header.cache-on-sensitive`	敏感响应缺少 Cache-Control: no-store	OWASP WSTG-ATHN-06, CWE-525
`request.sensitive-param-in-url`	查询串携带会话类参数	OWASP WSTG-ATHN-05, CWE-598
`request.get-with-body`	GET/HEAD 请求带正文	RFC 9110 §9.3.1
`request.cookie-over-http`	明文 HTTP 上传输 Cookie	OWASP WSTG-SESS-02, CWE-319
`request.duplicate-param`	同名查询参数重复且值不同	OWASP WSTG-INPV-04, CWE-235
`header.x-xss-protection-legacy`	X-XSS-Protection 处于启用态（该头已被现代浏览器废弃）	OWASP Secure Headers Project: X-XSS-Protection
`header.csp-missing`	HTML 响应完全没有 CSP	MDN: Content-Security-Policy
`header.csp-wildcard-allowlist`	CSP 的关键指令里出现裸 *（无方案限制）	MDN: CSP source values（* 通配）
`header.csp-report-only-only`	只有 Report-Only，没有强制生效的 CSP	MDN: Content-Security-Policy-Report-Only
`header.missing-permissions-policy`	HTML 响应缺少 Permissions-Policy	MDN: Permissions-Policy
`header.missing-coop`	HTML 响应缺少 Cross-Origin-Opener-Policy	MDN: Cross-Origin-Opener-Policy
`header.missing-coep`	HTML 响应缺少 Cross-Origin-Embedder-Policy	MDN: Cross-Origin-Embedder-Policy
`header.missing-corp`	HTML 响应缺少 Cross-Origin-Resource-Policy	MDN: Cross-Origin-Resource-Policy
`header.referrer-policy-unsafe`	Referrer-Policy 的取值比不写更差（unsafe-url 等）	MDN: Referrer-Policy, OWASP Secure Headers Project: Referrer-Policy
`header.cross-domain-policies-not-none`	X-Permitted-Cross-Domain-Policies 存在但不是 none	OWASP Secure Headers Project: X-Permitted-Cross-Domain-Policies

已知局限

1. curl -v 不承载正文：适配器按「无正文」处理，并把 [data not shown] 记进 notes。
2. raw 的请求/响应分界按声明长度消费后再找状态行；正文里恰有一行以 HTTP/ 开头且声明长度不可用时，可能误判分界。
3. HAR 不补 Content-Length：源报文没有定界信息时正文只存在于原始字节里，因此依赖定界的规则（如请求带正文）不会触发。
4. HTTP/2、HTTP/3 的版本串会被归一（`h2`/`h3`/`h2c`/`HTTP/2` → `HTTP/2.0` 等）才能通过请求行校验；本工具不实现长连接与帧层语义（HAR 里的 h2 头已经是解码后的明文头）。
5. Burp 的 <url>/<host>/<mimetype>/<extension>/<comment> 只进 notes，不参与判定。
6. 正文为 gzip/deflate/br/zstd 压缩时不解压，正文相关判据记 not-applicable。

7. 证据以规范化文本（canonical_text）为基准，不是原始文件；HAR 等格式的原文里找不到头行原文属正常。
8. 本工具只列信号，不下漏洞结论，也不发起任何网络请求。